Chat com ordenação

Relatório – esboço inicial

Levi Henri André Gabriel

5 de novembro de 2026

Estado. Este arquivo organiza o relatório pedido no enunciado. A versão final deverá ter 4 a 6 páginas, com a solução realmente implementada, as decisões efetivas e os testes executados. A extensão atual é provisória. Em 5/11 estão implementados transporte, ordenação, heartbeat, Bully e bootstrap (T02–T04). Em 6/11, duas correções de robustez foram feitas e o Compose foi executado nesse escopo. Reorganização, reentrada e painel seguem pendentes.

1 Objetivo e arquitetura proposta

Construir um chat cuja mesma sequência de mensagens seja entregue aos clientes conectados a nós diferentes, com recuperação após a queda de um nó.

Parte	Proposta inicial
Aplicação	Python e FastAPI; interface em HTML, CSS e JavaScript puro.
Execução	Três nós equivalentes no Docker Compose, inicialmente em uma máquina.
Transporte	WebSocket para clientes; HTTP e JSON entre nós.
Descoberta	DNS do Compose e nomes de serviços fornecidos por configuração.
Ordenação	Multicast total ordenado por Lamport, com ACK de todos na visão.
Coordenação	Heartbeat e Bully para escolher quem reorganiza o grupo.
Estado	Histórico e pendências em memória, com cópias em cada nó.

Princípios. Funções pequenas, código explicável pelos quatro integrantes e dependências justificadas. Começar com uma sala, apelido e texto. O único extra planejado é um painel com nós, líder, época e relógios. A demonstração em uma máquina é uma proposta, ainda a alinhar com o grupo.

2 Hipóteses e objetivos de correção

O cenário inicial admite uma queda por vez, comunicação funcionando entre os sobreviventes e nenhuma nova falha durante reorganização ou sincronização. Os timeouts pressupõem respostas dos nós saudáveis dentro dos limites configurados.

Pretende-se demonstrar: mesma ordem de entrega, ausência de duplicatas no histórico, preservação das mensagens confirmadas após a falha prevista e sincronização de um nó que retorna. Uma tentativa sem confirmação pode ter sido entregue.

Partições e falsas suspeitas persistentes ficam fora da garantia inicial. São necessários pelo menos dois dos três nós para operar. Essa regra não prova ausência de líderes concorrentes nem permite classificar o sistema como CP. Reiniciar todos os nós perde o histórico.

3 Algoritmos e decisões de projeto

Problema	Escolha e justificativa inicial
Ordem de entrega	Multicast por Lamport, fila ordenada e ACKs de todos na visão. A ordenação é responsabilidade do protocolo distribuído de mensagens.
Detecção de falha	Heartbeat com timeout. O timeout produz suspeita; a reorganização é uma etapa separada.
Coordenação	Bully e época identificam o coordenador da visão. O líder administra o grupo e não numera as mensagens do chat.
Recuperação	Pausa, reconciliação de estado e instalação confirmada da visão. A segurança dessa adaptação didática ainda precisa ser examinada.
Reentrada	Snapshot antes de admitir o nó e permitir sua participação na eleição.

O contrato de mensagens e as transições propostas estão em `docs/protocolo.tex`. O relatório deve explicar escolhas e consequências, sem reproduzir todos os envelopes do protocolo.

A configuração pequena e o estado em memória reduzem dependências e facilitam a explicação. Em contrapartida, o histórico depende dos sobreviventes. A exigência de ACK de todos torna a entrega dependente do membro mais lento; uma suspeita de falha interrompe o fluxo até a reorganização.

A troca de visão usa pausas locais para coletar histórico e pendências dos sobreviventes. Cada pausa define uma fronteira de processamento, não um instante global. O histórico reconciliado deve preservar como prefixo o que cada nó já entregou. Os detalhes e um exemplo com mensagens em trânsito ficam no protocolo.

4 Consistência e disponibilidade: análise inicial

A consistência pretendida é a mesma ordem de mensagens entregues e um histórico anterior preservado como prefixo após a troca de visão. Isso ainda é um objetivo, não uma propriedade demonstrada da implementação.

Durante eleição, instalação de visão e sincronização, a proposta pausa a aplicação. Com um único sobrevivente, novas mensagens ficam indisponíveis. A opção prioriza preservar o estado conhecido dentro do cenário de falha suportado.

A regra de dois participantes não prova exclusividade de liderança ou consistência sob partições. A ordenação total proposta também não estabelece, por si só, linearizabilidade. A análise final de CAP deverá explicitar o modelo de falha, a semântica de confirmação e o comportamento observado, sem atribuir ao sistema garantias de partição que não foram implementadas e verificadas. Como base para essa discussão: Gilbert e Lynch, Perspectives on the CAP Theorem.

5 Validação planejada e resultados

Na versão final, registrar condições, comandos, resultado real e evidência de cada teste selecionado. A coluna abaixo ainda descreve o esperado; os resultados parciais estão no parágrafo seguinte.

Cenário planejado	Comportamento esperado
Concorrência	Clientes em nós diferentes recebem a mesma sequência.
Retry	Repetir um pacote após resposta perdida não repete seus efeitos.
Reconexão	Reenvio com o mesmo UUID não duplica o histórico.
Queda do líder	Mensagens confirmadas sobrevivem e os nós restantes retomam.
Mensagens em trânsito	A troca de visão preserva o prefixo já entregue.
Reentrada	O nó reiniciado sincroniza antes de participar normalmente.
Um sobrevivente	O sistema interrompe a aceitação de novas mensagens.

Resultados atuais (5/11, `pytest em tests/`). Concorrência e retry: em uma simulação em memória com 200 sementes, que intercala canais ao acaso, mantém FIFO em cada canal e repete pacotes, os três nós entregaram a mesma sequência, sem perdas nem duplicatas. Defeitos injetados de propósito fizeram esses testes falharem. Com três processos reais em `localhost`, três clientes WebSocket enviaram 45 mensagens concorrentes e receberam a mesma sequência. O mesmo UUID enviado a dois nós entrou uma vez no histórico. Queda do líder: ao matar o processo do nó 3, os nós 1 e 2 suspeitaram após o timeout, elegeram o nó 2 por Bully e continuaram recusando novos envios, como previsto até a reorganização (T05). Mensagens em trânsito, reentrada e um sobrevivente ainda não foram testados.

Correções e resultados de 6/11. O canal FIFO passou a repetir o mesmo pacote também após HTTP 500 e a parar, sem deixar o seguinte ultrapassá-lo, diante de uma recusa permanente; antes, abandonava o pacote e enviava o próximo. Os pacotes entre nós passaram a ter envelope e campos validados antes de qualquer efeito; antes, um `MSG` sem `pacote_id` alterava fila e relógio e só depois lançava exceção. Com `pytest`, 289 testes passaram em três execuções (Python 3.13, macOS).

- Testes controlados: 7 do `CanalFIFO` real com HTTP simulado (500 seguido de sucesso, 503, conexão e timeout, resposta perdida com receptor real, recusa permanente, independência entre destinos) e 40 de validação (37 pacotes malformados recusados com 400, sem alterar relógio, fila, ACKs, histórico, deduplicação ou pacotes emitidos, e formatos válidos, inclusive época `null` nos controles). Contra o código anterior, os casos de 500 e de recusa permanente falharam, assim como 26 dos 37 pacotes malformados.
- Integração local: os 4 testes com três processos `uvicorn` passaram.
- Compose (OrbStack, Docker 29.8.2), com imagens reconstruídas: os três nós ficaram ativos na visão $(1, 3)$; três clientes WebSocket, um por nó, enviaram 20 mensagens concorrentes cada e receberam as mesmas 60 entregas na mesma ordem, com `/historico` igual nos três nós. O mesmo UUID enviado aos nós 1 e 2 e reenviado ao 1 apareceu uma vez em cada histórico. Após `docker compose kill no3`, os nós 1 e 2 elegeram o nó 2 e ficaram reorganizando na época $(1, 3)$, recusando novo envio, com os históricos de 61 mensagens preservados e iguais.

Revisão do esboço. Foi examinado um exemplo com queda do nó 3: o nó 1 entregou uma mensagem que ainda era pendência do nó 2, e uma segunda mensagem estava em trânsito para o nó 2. Um modelo de dados em memória conferiu a união dos relatos, a preservação de prefixos, os sufixos a notificar e o bloqueio diante de um prefixo incompatível. Não houve execução dos nós, da rede ou do protocolo; essa conferência não demonstra sua correção geral.

6 Limitações e pontos a consolidar

Bootstrap, formatos, confirmação ao cliente e deduplicação entre nós foram fechados na v1 do protocolo. O bootstrap exige os três nós. Ainda é necessário definir a fronteira de congelamento e a reconciliação. Partições, falsas suspeitas persistentes e novas falhas durante reorganização ficam fora da garantia inicial. As limitações efetivas devem ser reavaliadas conforme a implementação e os experimentos.

7 Uso de IA

Até esta versão, ferramentas de IA auxiliaram o planejamento, a revisão das hipóteses do protocolo e a elaboração dos esboços de documentação. O plano foi discutido com Codex e revisado com apoio de Claude. As correções de 6/11 no canal FIFO e na validação de pacotes, com seus testes, foram escritas com Claude Code a partir de um briefing do grupo; a revisão humana dessas correções está pendente. Este registro deve ser atualizado com os usos posteriores e a revisão humana realizada pelo grupo. Os integrantes precisam compreender e explicar a solução.

Referências iniciais

Enunciado do professor: Trabalho Final – Sistemas Distribuídos 2026, fornecido durante o planejamento. Lamport (1978); Garcia-Molina (1982); Van Renesse et al., comunicação e mudança de visão.